

Lernziele:

- Vertiefung: Assembler-Code für Funktionsdefinitionen und Funktionsaufrufe und Schleifen

Zielsetzung

In dieser Session soll der Compiler so erweitert werden, dass in unserer Programmiersprache Funktionen definiert und aufgerufen werden können. Dafür wurde in der letzten Session der Code-Generator sauber vorbereitet.

In dieser Session soll aber auch angedeutet werden, dass es in der Praxis nicht immer so läuft, dass man direkt mit einem sauberen Design anfängt. Denn dazu muss man erst einmal wissen, was man alles braucht und wie man das Ganze dann schön säuberlich in Komponenten auslagert. Mit einem "Dirty-Hacker-Approach" werden zu diesem Zweck einfache While-Schleifen unterstützt.

Schritt 1 Lexer

Unser Lexer sollte auch Schlüsselwörter wie `fn`, `while`, `if`, `else` erkennen. Für Anweisungsblöcke sollen außerdem auch `{` und `}` erkannt werden. Dazu kann man zunächst in `lexer.h` neue Enum-Konstanten deklarieren (`LBRACE` wird für `{` und `RBRACE` für `}` verwendet):

```
enum TokenKind
{
    // ...
    LBRACE,
    RBRACE,
    FN,
    WHILE,
    IF,
    ELSE,
};
```

Die zusätzlichen Klammern zu unterstützen sollte kein Problem sein; man macht das analog zu Zeichen wie `*` oder `+`. Um die Schlüsselwörter zu erkennen, bin ich wenig elegant, aber dafür effektiv vorgegangen und habe die Funktion `getToken` wie folgt modifiziert:

```
fn getToken(): TokenKind
{
    getToken_();
    token.val = UStrCreate(val);
    if (token.kind == IDENTIFIER) {
        if (!strcmp(token.val, "fn")) {
            return token.kind = FN;
        } else if (!strcmp(token.val, "while")) {
            return token.kind = WHILE;
        }
    }
    return token.kind;
}
```

Im alten Code wird `getToken_` aufgerufen, um Token wie Identifier oder spezielle Zeichen zu erkennen. Dann wird die zugehörige Zeichenkette `val` mit `UStrCreate` in die Unique-String-Datenstruktur eingecheckt. Im neuen Code wird jetzt einfach überprüft, ob ein Identifier nicht doch ein Schlüsselwort ist. In so einem Fall wird das Token dann sozusagen "gepatcht". Dabei werden Strings mit `strcmp` verglichen, also `string.h` einbinden und daran denken, dass `strcmp` eine Null liefert, wenn Strings gleich sind (deshalb das logische Nicht in der If-Bedingung).

Aufgabe:

Passt den Lexer an und testet ihn. Ändert den Code so ab, dass auch `"if"` und `"else"` erkannt werden.

Schritt 2 Expression-Tree

Zunächst werden wir keine Argumente unterstützen, was alles deutlich vereinfacht. In der Grammatik wird zum Beispiel ein Funktionsaufruf dadurch ausgezeichnet, dass nach einem Identifier ein leeres Klammerpaar kommt. Das heißt `bar()` ist ein Funktionsaufruf, aber `foo` eine Variable. Bevor wir uns jedoch an die Grammatik und den Parser machen, bereiten wir zunächst die Datenstruktur für Expression-Trees vor.

In `expr.hdr` wird für `ExprKind` eine neue Enum-Konstante `EXPR_CALL` deklariert:

```
enum ExprKind
{
    EXPR_BINARY,
    // ...
    EXPR_BINARY_END,

    EXPR_UNARY = EXPR_BINARY_END,
    // ...
    EXPR_UNARY_END,

    EXPR_PRIMARY = EXPR_UNARY_END,
    // ...
    EXPR_PRIMARY_END,

    EXPR_CALL,
};
```

Diese Kontenart gehört weder zu den Primary-, Unary- oder Binary-Expressions. Denn so ein Knoten hat im Allgemeinen als Childnodes einen Ausdruck für die Funktionsadresse und eine Liste mit den Argumenten. Da wir aber keine Argumente verwenden, vereinfacht sich das. Zudem werden wir zunächst keine Funktionszeiger unterstützen. Wenn man eine Funktion aufrufen will, dann muss man stets den Funktionsnamen angeben. Einen Knoten für den Funktionsaufruf kann man dazu mit der Funktion

```
extern fn createCallExpr(val: -> UStr): -> Expr;
```

erzeugen, die ebenfalls in `expr.hdr` deklariert wird. Das Testprogramm kann man wie folgt um einen Funktionsaufruf anpassen:

```
1 @ <stdio.hdr>
2 @ "expr.hdr"
3 @ "gen.hdr"
4
5 fn main()
6 {
7     local expr: -> Expr;
8
9     local _2: -> UStr = UStrCreate("2");
10    local _3: -> UStr = UStrCreate("3");
11    local _f: -> UStr = UStrCreate("f");
12    local _x: -> UStr = UStrCreate("x");
13
14    expr = createBinaryExpr(EXPR_SUB,
15                           createBinaryExpr(EXPR_DIV,
16                                             createIntegerExpr(_2),
17 createIntegerExpr(_3)),
18                           createCallExpr(_f));
19    printExprLatexTree(expr);
20    expr = createBinaryExpr(EXPR_ASSIGN,
21                           createDerefExpr(
22                               createAddrExpr(
23                                   createIdentifierExpr(_x))),
24                           expr);
25    printExprLatexTree(expr);
26    printf("\n");
27
28    genReleaseReg(loadExpr(expr));
29
30    releaseExpr(expr);
31 }
```

Aufgaben:

1. Passt den Header `expr.h` und das Testprogramm `xtest_expr.abc` an. Danach sollte der Compiler alles übersetzen und nur noch ein Linkerfehler auftreten.
2. Implementiert die Funktion `createCallExpr`. Dazu die Funktion `createIdentifier` könnt ihr dafür kopieren und wie folgt abändern:

```
fn createCallExpr(val: -> UStr): -> Expr
{
    local n: -> Expr = malloc(sizeof(*n));
    assert(n && "out of memory");

    n->kind = EXPR_CALL;
    n->identifizier = val;
    return n;
}
```

Danach sollte sich das Testprogramm in eine ausführbare Datei übersetzen lassen. Bei der Ausführung wird es jedoch noch zu einem Assertion-Failure kommen, da noch nicht alle Funktionen angepasst wurden.

3. Passt die Funktion `printExprTree` so an, dass die neue Knotenart unterstützt wird. Dazu könnt ihr folgende Struktur verwenden:

```
fn printExprTree(expr: -> Expr)
{
    if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
        // ..
    } else if (expr->kind >= EXPR_UNARY && expr->kind < EXPR_UNARY_END) {
        // ..
    } else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
        // ..
    } else if (expr->kind == EXPR_CALL) {
        printf(" [ %s() ]", expr->identifizier);
    } else {
        assert(0);
    }
}
```

Analog kann man auch `loadExpr` anpassen:

```
fn loadExpr(expr: -> Expr): GenReg
{
    if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
        // ...
    } else if (expr->kind >= EXPR_UNARY && expr->kind < EXPR_UNARY_END) {
        // ...
    } else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
        // ...
    } else if (expr->kind == EXPR_CALL) {
        genPrepareFunctionCall(0);
        return genExecuteFunctionCall(expr->identifizier, 0);
    }
    assert(0 && "loadExpr: not all cases handled!");
    return genConstant("0");
}
```

Hier werden vom Code-Generator die Funktionen verwendet, die Assemblercode für einen Funktionsaufruf erzeugen.

Zur Kontrolle: Im LaTeX-Code sollte für den Expression-Tree folgender Code stehen:

```
[ {=} [ * [ \& [ x ] ] ] [ - [ / [ 2 ] [ 3 ] ] [ f() ] ] ]
```

Der Assemblercode sollte so aussehen, wie rechts dargestellt. Bitte genau vergleichen, eventuell war da im letzten Aufgabenblatt noch ein Bug.

```
load 2, %4
load 3, %5
div %5, %4, %4
sub 8*(3 + 0), %SP, %SP
call f, %3
movq retval(%SP), %5
add 8*(3 + 0), %SP, %SP
sub %5, %4, %4
load x, %5
movq %4, (%5)
```

Schritt 3 Parser

Die Grammatik wird wie folgt erweitert:

```
prog = { function }
function = "fn" identifier compound-statement
compound-statement = "{" { (compound-statement | expr-statement) } "}"
expr-statement = expr ";"
```

```
expr = additive { "=" expr }
additive = term { ( "+" | "-" ) term }
term = unary { ( "*" | "/" ) unary }
unary = ( ( "*" | "&" ) unary )
        | factor
factor = decimal-literal
        | identifier [ "(" ")" ]
        | "(" expr ")"
```

Ein Programm besteht also aus einer beliebigen Anzahl von Funktionsdefinitionen. Eine Funktionsdefinition beginnt mit dem Schlüsselwort `fn` gefolgt vom Funktionsnamen und einem Compound-Statement (Anweisungsblock). Ein Compound-Statement wiederum kann beliebig viele Statements (Anweisungen) enthalten, wobei ein Statement entweder ein Expression-Statement oder ein Compound-Statement ist. Die Funktionsaufrufe wurden hier in der Produktionsregel für einen Faktor eingebaut. Wenn nach einem Identifier ein leeres Klammerpaar kommt, dann ist dies ein Funktionsaufruf. Das geht gut, da wir keine Funktionszeiger unterstützen.

Deklariert in `parser.hdr` zusätzlich:

```
extern fn parseFunction(): bool;
```

Passt das Programm `xtest_abc.abc` wie folgt an:

```
@ <stdio.hdr>
@ "gen.hdr"
@ "genvar.hdr"
@ "lexer.hdr"
@ "parser.hdr"

fn main()
{
    genInit();
    getToken();
    // parse sequence of functions
    while (parseFunction()) {
    }
    if (token.kind != EOI) {
        printf("%u.%u: syntax error\n", token.pos.row, token.pos.col);
    }
    genHalt(3);
    genPrintSymtab();
}
```

Bis auf einen Linkerfehler sollte dann bereits alles übersetzen. In `parser.abc` kann man `parseFactor` den Code so anpassen, dass bei einem Identifier entweder ein Knoten mit `createCallExpr` oder `createIdentifierExpr` erzeugt wird:

```
} else if (token.kind == IDENTIFIER) {
    local val: -> UStr = token.val;
    getToken();
    if (token.kind == LPAREN) {
        getToken();
        if (token.kind != RPAREN) {
            printf("%u.%u: ')' expected\n", token.pos.row, token.pos.col);
            return nullptr;
        }
        getToken();
        return createCallExpr(val);
    } else {
        return createIdentifierExpr(val);
    }
}
```

Die neuen Parse-Funktionen kann man nun wie üblich programmieren. Wegen der gegenseitigen Abhängigkeiten der Produktionsregeln sind dabei Forward-Deklarationen notwendig. Im Sourcefile `parser.abc` habe ich zum Beispiel als Erstes die Funktion `parseCompoundStatement` implementiert und deshalb davor die Funktion `parseExprStatement` deklarieren müssen:

```
fn parseExprStatement(): bool;

fn parseCompoundStatement(): bool
{
    if (token.kind != LBRACE) {
        return false;
    }
    getToken();
    while (parseExprStatement() || parseCompoundStatement()) {
    }
    if (token.kind != RBRACE) {
        printf("%u.%u: '}' expected\n", token.pos.row, token.pos.col);
        return false;
    }
    getToken();
    return true;
}
```

Danach habe ich die Funktion `parseFunction` implementiert:

```
fn parseFunction(): bool
{
    if (token.kind != FN) {
        return false;
    }
    getToken();
    if (token.kind != IDENTIFIER) {
        printf("%u.%u: function name expected\n", token.pos.row, token.pos.col);
        return false;
    }
    genBeginFunctionDefinition(token.val);
    getToken();
    if (!parseCompoundStatement()) {
        printf("%u.%u: compound expression expected\n",
            token.pos.row, token.pos.col);
        return false;
    }
    genEndFunctionDefinition();
    return true;
}
```

Und dann `parseExprStatement`:

```
fn parseExprStatement(): bool
{
    local expr: -> Expr = parseExpr();
    if (!expr) {
        return false;
    }
    if (token.kind != SEMICOLON) {
        printf("%u.%u: ';' expected\n", token.pos.row, token.pos.col);
        return false;
    }
    getToken();
    genReleaseReg(loadExpr(expr));
    return true;
}
```

Damit sollten einfache Funktionen ohne Parameter und ohne lokale Variablen unterstützt sein. Der Rückgabewert einer Funktion ist dabei der Wert des letzten Ausdrucks. In folgendem Beispiel hat die Funktion `foo` deshalb stets den Rückgabewert `41`. Als erste Funktion wird immer die Funktion `main` aufgerufen (egal an welcher Stelle im Programm diese definiert wurde) und deren Rück-

gabewert wird als Exit-Code verwendet:

```
MCL:s26 lehn$ cat test
fn main
{
    x = foo() + 1;
}

fn foo
{
    41;
}

MCL:s26 lehn$ ./xtest_abc < test > foo.s
MCL:s26 lehn$ ulmas foo.s && ulm a.out ; echo $?
42
```

In unserer Programmiersprache müssen Funktionen nicht deklariert werden bevor man sie aufruft. Testet, dass man `foo` auch vor `main` definieren kann. Der Nachteil ist, dass es keine Fehlermeldungen gibt, wenn man eine Funktion aufruft, die überhaupt nicht definiert wird. Ruft zum Beispiel `bar` statt `foo` auf. Ein Fehler tritt erst auch, wenn man versucht `a.out` auszuführen, da das Symbol `bar` nicht definiert ist (würden wir einen Linker verwenden, dann wäre der Fehler etwas früher entdeckt worden).

Etwas enttäuschend ist es, wenn obiges Programm wie folgt modifiziert wird:

```
MCL:s26 lehn$ cat test
fn main
{
    x = 1 + foo();
}

fn foo
{
    41;
}

MCL:s26 lehn$ ./xtest_abc < test > foo.s
MCL:s26 lehn$ ulmas foo.s && ulm a.out ; echo $?
82
```

Schaut man sich den erzeugten Assemblercode an, dann sieht man, dass für den Ausdruck

```
x = 1 + foo();
```

eigentlich korrekter Code erzeugt wird. Zunächst wird die Konstante `1` in das Register `%4` geladen, dann wird die Funktion `foo` aufgerufen und deren Rückgabewert in das Register `%5` geladen. Das Problem ist, dass innerhalb der Funktion `foo` das Register `%4` verwendet wird.

Um das Problem kümmern wir uns aber nicht heute. Vorerst gehen wir dem Problem einfach aus dem Weg, indem wir Funktionsaufrufe isoliert verwenden:

1. Ausdrücke, die nur aus einem Funktionsaufruf bestehen, zum Beispiel

```
foo();
```

2. Oder Ausdrücke, bei denen rechts vom Gleichheitszeichen ausschließlich ein Funktionsaufruf steht:

```
x = foo();
```

Das Problem kann man aber später leicht beheben.

Schritt 4: Integrierte Funktionen für die Ein- und Ausgabe

Funktionen für das Einlesen und Ausgeben eines einzelnen Zeichens sind plattformabhängig und müssen entweder vom Betriebssystem (was wir auf der ULM nicht haben) zur Verfügung gestellt werden oder direkt in Assembler (was wir machen) programmiert werden.

Ändert in `gen.abc` die Funktion wie folgt:

```
fn genInit()
{
    printf("\t.equ SP,      1\n");
    printf("\t.equ FP,      2\n");
    printf("\t.equ retval,  16\n");
    printf("\t.text\n");
    printf("\tload 0,      %%SP\n");
    genPrepareFunctionCall(0);
    local reg: GenReg = genExecuteFunctionCall("main", 0);
    printf("\thalt    %%d\n", reg);
    genReleaseReg(reg);

    genBeginFunctionDefinition("getchar");
    printf("\tgetc    %%4\n");
    genEndFunctionDefinition();

    genBeginFunctionDefinition("putchar");
    printf("\tload    PUTCHAR,    %%3\n");
    printf("\tmovq    (%%3),    %%3\n");
    printf("\tputc    %%3\n");
    genEndFunctionDefinition();

    genAddGlobalVar(UStrCreate("PUTCHAR"));
}
```

Damit werden in jedem Programm intern die Funktionen `getchar` und `putchar` für die Ein- und Ausgabe eines einzelnen Zeichens definiert. Außerdem wird eine globale Variable `PUTCHAR` angelegt, die verwendet werden kann, um der Funktion `putchar` einen Parameter (auf Umwegen) zu übergeben. Folgendes Programm demonstriert, wie das gemeint ist:

```
MCL:s26 lehn$ cat test2
fn main
{
    x = getchar();
    x = x + 1;
    PUTCHAR = x;
    putchar();
    PUTCHAR = 10;
    putchar();
}
MCL:s26 lehn$ ./xtest_abc < test2 > foo.s
MCL:s26 lehn$ ulmas foo.s && ulm a.out
A
B
```

Bevor man die Funktion `putchar` aufruft, setzt man die (globale) Variable `PUTCHAR` auf den ASCII-Wert, der ausgegeben werden soll. Der ASCII-Wert für einen Zeilenumbruch ist zum Beispiel `10`.

Schritt 5: While-Schleifen

Die Grammatik wird wie folgt erweitert:

```
prog = { function }
function = "fn" identifier compound-statement
compound-statement = "{" { (compound-statement | while-statement | expr-statement) } "}"
while-statement = "while" "(" expr ")" compound-statement
```

Neu hinzugekommen ist die Produktionsregel für ein While-Statement. Hier gehen wir mit dem Dirty-Hacker-Approach an die Sache. Der Parser wird teilweise direkt mit `printf`-Anweisungen Code ausführen, also Dinge machen, die man eigentlich im Code-Generator hinter Funktionen verstecken sollte (was man auch später machen sollte).

Der Hack für die zusätzliche Parse-Funktion ist vorgegeben und eine eurer Aufgaben ist es herauszufinden, nach welchem Prinzip diese funktioniert:

```

60 fn parseWhileStatement(): bool
61 {
62     static count: int = 0;
63     if (token.kind != WHILE) {
64         return false;
65     }
66     getToken();
67     ++count;
68     printf("while%d:", count);
69     if (token.kind != LPAREN) {
70         printf("%u.%u: '(' expected\n", token.pos.row, token.pos.col);
71         return false;
72     }
73     getToken();
74     local expr: -> Expr = parseExpr();
75     if (!expr) {
76         printf("%u.%u: expression expected\n", token.pos.row, token.pos.col);
77         return false;
78     }
79     if (token.kind != RPAREN) {
80         printf("%u.%u: ')' expected\n", token.pos.row, token.pos.col);
81         return false;
82     }
83     getToken();
84     local reg: GenReg = loadExpr(expr);
85     printf("\tsub %%0, %%d, %%0\n", reg);
86     printf("\tjz done%d\n", count);
87     genReleaseReg(reg);
88
89     if (!parseCompoundStatement()) {
90         printf("%u.%u: compound expression expected\n",
91             token.pos.row, token.pos.col);
92     }
93     printf("\tjmp while%d\n", count);
94     printf("done%d:\n", count);
95     return true;
96 }

```

Passt außerdem die Funktion `parseCompoundStatement` entsprechend der Grammatik an.

In der Parse-Funktion für ein While-Statement wird Assembler-Code erzeugt, der Sprunganweisungen enthält. Aktuell werden die von unserer Architektur aber noch nicht unterstützt. Fügt in `simple.isa` am Anfang der Datei folgendes Format hinzu:

```
REL_JMP      (OP u 8) (offset j 24)
```

Ergänzt die Datei dann am Ende noch um die Beschreibung für den unbedingten Sprung `jmp` und den bedingten Sprung (falls ZF = 1 ist) `jz`:

```

0x40 REL_JMP
:    jmp offset
    ulm_unconditionalRelJump(offset);

0x41 REL_JMP
:    jz offset
    ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 1, offset);

```

Erzeugt eine neue virtuelle Maschine und einen neuen Assembler für die erweiterte ISA:

- Löscht das Unterverzeichnis `simple` mit dem Kommando `rm -rf simple`
- Erzeugt die Tools mit `ulm-generator --install simple.isa`

Mit dem folgenden Programm kann man eine Zahl eingeben, von der die Fakultät berechnet wird, die dann als Exit-Code (Modulo 256) verwendet wird.

```

fn main
{
    n = 0;
    while ((d = getchar()) - 10) {
        n = n * 10 + d - 48;
    }
    factorial();
}

fn factorial
{
    res = 1;
    while (n) {
        res = res * n;
        n = n - 1;
    }
    res;
}

```